# Grammar and Parsing

CSE

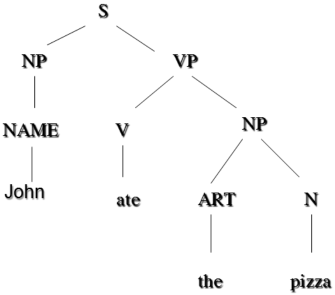
Ho Chi Minh City University of Technology

2023.01

# Outline

# Grammar and Sentences Structure

The most common method to study the structure of sentence is how sentence is broken into its major subparts and how those subparts are broken up in turn, is as a tree.




Hình: (3.1) Syntactic structure of sentence "John ate the pizza"

# Grammar and Sentences Structure



S consists initial noun phrase NP and verb phrase VP. initial NP is made of NAME *John*. Initial VP is composed of verb *ate* and NP, which consists ART *the* and N *pizza*
The structure of sentence may be represented in another way:

# Context Free grammar (CFG)



$G = (S, P, N, T)$
$S$: start symbol $S$,
$P$: production rules, which have the form: $A \rightarrow \alpha$;
$N$, $T$: set of lexical symbols (word categories).
$N$ is set of non- terminal symbols; $T$ is set of terminal symbols.
There are two important process based on derivations: sentence **generation** and sentence **parsing**
There are two methods of search (structure of sentence): Top-down and Bottom-up.

# What makes a Good Grammar



To construct a grammar for language, we are interested in generality, the range of sentences the grammar analyzes correctly; selectivity, the range of non-sentences it identifies as problematic; and understandability, the simplicity of grammar itself

Beginning with small grammar, such as those that describe only few types of sentences, one structural analysis of a sentence may appear as understandable as another.

Then we attempt to extend a grammar to cover a wide range of sentences, however, we often find one analysis is easily extendable, while the other requires complex modification.

# What makes a Good Grammar (cont.)



- This analysis retains its simplicity and generality as it is extended is more desirable.
- To pay close attention to the way the sentence is divided into its subparts, called constituents.
  By using our intuition we can apply specific tests, as follows.
  - To decide that a group of words forms a particular constituent;
  - Try to construct a new sentence that involves that group of words in conjunction with another group of words classified as the same type of constituent.

Example:

- I ate a hamburger and a hot dog (NP-NP).
- I will eat the hamburger and throw away the hot dog (VP-VP)

# Top-down Parser



- Parsing algorithm is a procedure that searches through various ways of grammar rules to find a combination that generates a tree that could be the structure of the input sentence.
- A top-down parser starts with the $S$ symbol and attempts to rewrite it into a sequence of terminal symbols that matches the classes (categories) of the words in the input sentence.

# A Simple Top-Down Parsing Algorithm

The algorithm starts with the initial state ((S) 1) and no backup

1. Select the current state: take the first state of the possibilities list and call it C. If the possibilities list is empty, then the algorithm fails ( no successful parse is possible)
2. If C consists of an empty list and the position of word is at the end of sentence, then the algorithm is succeed
3. Otherwise, generate the next possible states.
   - 3.1 If the first symbol on the symbol list C is terminal (lexical symbol), and the next word in the sentence can be in that class, then create a new state by removing the first symbol from the symbol list C, update the word position and add it to the possibilities list
   - 3.2 If the first symbol of C is non-terminal, generate a new state of each rule in the grammar that can rewrite that non-terminal symbol and add them all to the possibilities list.

# A Simple Top-Down Parsing Algorithm

Example: Parse the sentence: $_1$the $_2$dogs $_3$cried$_4$

Grammar:

1. S $\rightarrow$ NP VP
2. NP $\rightarrow$ ART N
3. NP$\rightarrow$ ART ADJ N
4. VP $\rightarrow$ V
5. VP $\rightarrow$ V NP

# A Simple Top-Down Parsing Algorithm (continue)



| Step | current state | back state | note |
|---|---|---|---|
| 1 | ((S)1) | | initial position |
| 2 | ((NP VP)1) | | rewriting S by rule 1 |
| 3 | ((ART N VP)1) | | rewriting NP by rules 2&3 |
| | | (( ART ADJ N VP)1) | |
| 4 | ((N VP)2) | | matching ART with *the* |
| | | (( ART ADJ N VP)1) | |
| 5 | (( VP)3) | | matching N with *dogs* |
| | | (( ART ADJ N VP)1) | |
| 6 | ((V)3) | | rewriting VP by rules 4&5 |
| | | ((V NP)3)<br>(( ART ADJ N VP)1` | |
| 7 | (( )4) | | The parser succeeds as maching V with ***cried***, leaving an empty grammatical symbol list with an empty input sentence |

Hình: Top-Down Depth first parser of "The dog cried"

# Parsing as a Search Procedure



The top-down parser is described as a search procedure that implements as follows.
The possibilities list is initially set to the start state S of the parser.

1. Select the first state from possibilities list and remove it from the list.
2. Generate new states from a current state by trying every possible option from the selected (there may be none if we on a bad path).
3. Add the states generated in step 2 to the possibilities list and repeat step 1.

# Parsing as a Search Procedure

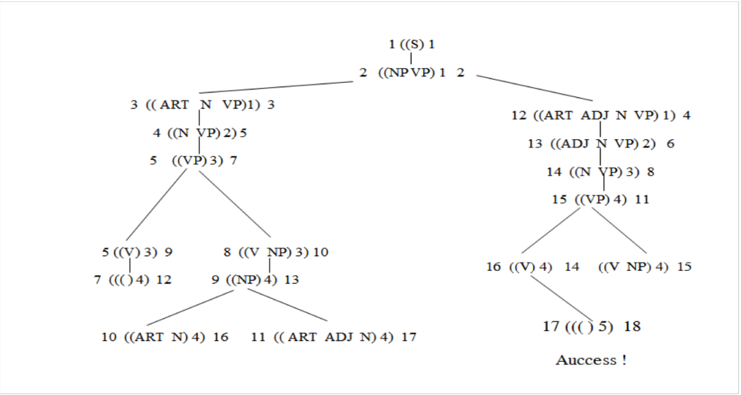


| Step | Current state | Backup state | comment |
|---|---|---|---|
| **01** | ((S) 1) | | |
| **02** | ((NP VP) 1) | | S rewritten to NP VP |
| **03** | ((ART N VP) 1) | ((ART ADJ N VP) 1) | NP rewritten producing two new states |
| **04** | ((N VP) 2) | ((ART ADJ N VP) 1) | |
| **05** | ((VP) 3) | ((ART ADJ N VP) 1) | The backup state remains |
| **06** | (( V) 3) | ((V NP) 3)<br>((ART ADJ N VP) 1) | |
| **07** | (( ) 4) | ((V NP) 3)<br>((ART ADJ N VP) 1) | |
| **08** | ((V NP) 3) | ((ART ADJ N VP) 1) | The first backup is chosen |
| **09** | ((NP) 4) | ((ART ADJ N VP) 1) | |
| **10** | (( ART N) 4) | ((ART ADJ N) 4)<br>((ART ADJ N VP) 1) | Looking for ART at 4 fails |
| **11** | ((ART ADJ N) 4) | ((ART ADJ N VP) 1) | Fails again |
| **12** | ((ART ADJ N VP) 1) | | Now exploring backup state saved in step 3 |
| **13** | (( ADJ N VP) 2) | | |
| **14** | (N VP) 3) | | |
| **15** | (VP) 4) | | |
| **16** | ((V) 4) | ((V NP) 4) | |
| **17** | (( ) 5) | | Success |

Hình: A Top Down Parse of "The old man cried"

# Parsing as a Search Procedure






Hình: Depth-first strategy and breadth-first strategy

# A bottom-Up Chart Parser



The main difference between top-down and bottom-up parsers is the way the grammar rules are used.

**The extension algorithm** Add a constituent C from position $p_1$ to $p_2$:

1. Insert C into chart from $p_1$ to $p_2$;
2. For any active arc of the form: $X \rightarrow X_1 ... \bullet C ... X_n$ from $p_0$ to $p_1$, add a new active arc $X \rightarrow X_1 ... C \bullet ... X_n$ from $p_0$ to $p_2$
3. For any active arc of the form: $X \rightarrow X_1 ... \bullet C$ from $p_0$ to $p_1$, add a new constituent of type X from $p_0$ to $p_2$ to the agenda.

# A Bottom-up Chart Parsing Algorithm



Do until there is no input left.

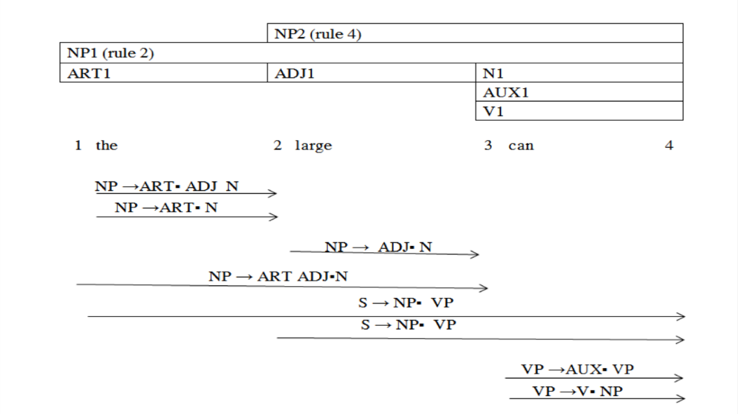
1. If the agenda is empty, then look up the interpretations (categories) of the new word in the input and add them to the agenda.
2. Select a constituent from the agenda (let's call it constituent C from $p_1$ to $p_2$).
3. For any grammar rule $X \to CX_1...X_n$, add active arc of the form $X \to \bullet CX_1...X_n$ from $p_1$ to $p_2$.
4. Add C to the chart by the extension algorithm.

---

Example: To parse a sentence "The large can can hold the water"

# A bottom-Up Chart Parser

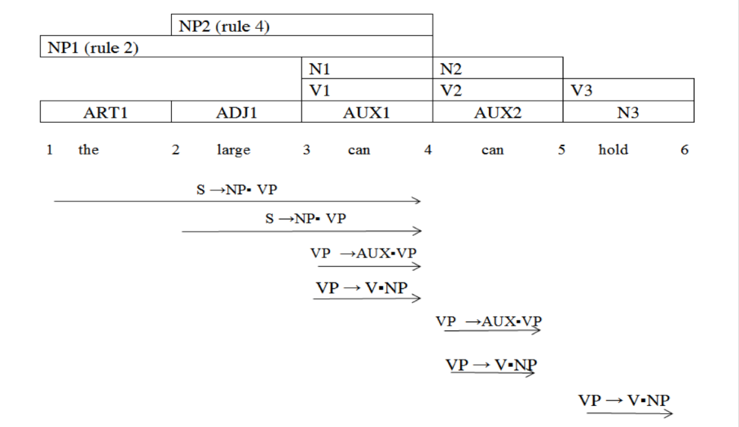





Hình: After parsing **the large can**

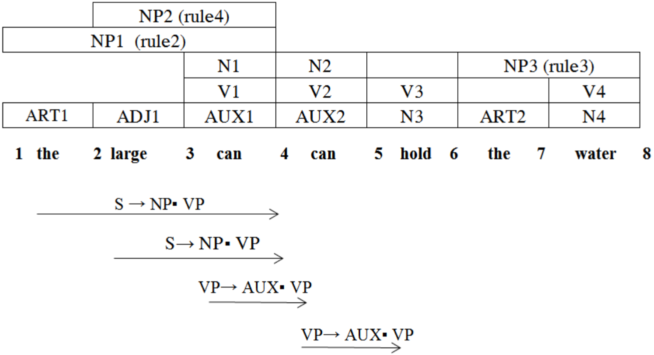
# A bottom-Up Chart Parser






Hình: The chart after adding **hold**, omitting arcs generated for the first NP

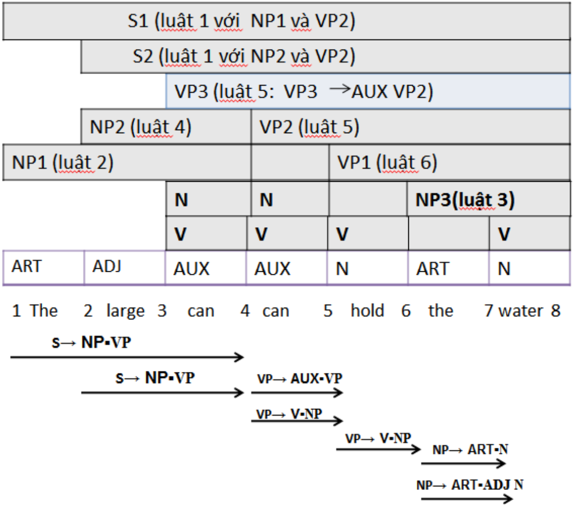
# A bottom-Up Chart Parser




Hình: The chart after the NPs are found, omitting all but the crucial active arcs

# A bottom-Up Chart Parser






Hình: The final chart

# Efficiency Considerations



- Chart–based parser can be considerably more efficient than parsers that rely only on a search because the same constituent never constructed more than once.
- The complexity of the pure top-down or bottom-up parser could require up to $C^n$ operations to parse a sentence of the length $n$, $C$ is a constant that depends on the specific algorithm we use.
- The complexity of the chart-based parser is $Kxn^3$. $K$ is a constant that depends on the algorithm, and $n$ is the sentence length.
- The chart-based parser would be up to many times faster than a pure parser.

# Top-Down Chart Parsing

So far we have seen a simple top-down method and a bottom-up chart-based method for parsing context-free grammars.

Now a new method is presented actually captures the advantages of both, that is **top-down chart parser**.

**Top-down arc Introduction Algorithm**

To add an arc $S \rightarrow C_1 ... \bullet C_i ... C_n$ ending at position $j$, do the following.

For any rule in the grammar of the form $C_i \rightarrow X_1 ... X_k$, recursively add new arc $C_i \rightarrow \bullet X_1 ... X_k$ from position j to j.

# Top-Down Chart Parsing Algorithm

**Initialization**: for every rule in the grammar of the form $S \to X_1 ... X_k$ add an arc; labeled $S \to \bullet X_1 ... X_k$ using the arc introduction algorithm.

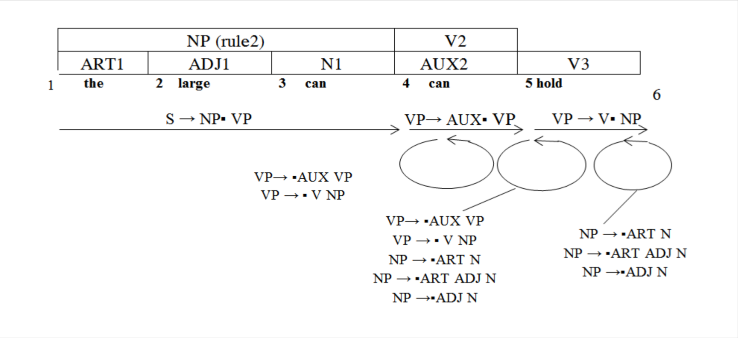
**Parsing**: Do until there is no input left

1. If agenda is empty, look up the interpretation of the next word and add them to the agenda.
2. Select constituent from the agenda (call it constituent C).
3. Using the arc extension algorithm, combine C with every active arc on the chart. Any new constituents are added to the agenda.
4. For any active arcs created in step 3, add them to the chart using the top-down arc introduction algorithm.

# Top-Down Chart Parsing



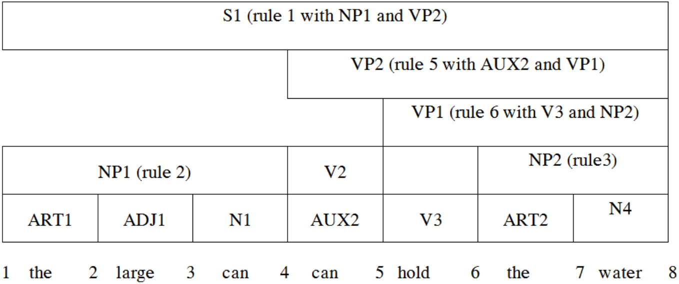
Example: $_1$the $_2$ large $_3$can $_4$can $_5$hold $_6$the $_7$water$_8$




Hình: The chart after adding hold, omitting arcs generated for the first NP

# Top-Down Chart Parsing

Example: $_1$the $_2$ large $_3$can $_4$can $_5$hold $_6$the $_7$water$_8$




Hình: The final chart for top-down filtering algorithm

# Probabilistic – or stochastic – context-free grammars (PCFGs)

G = (T, N, S, R, P)

- T is a set of terminal symbols
- N is a set of nonterminal symbols
- S is the start symbol ($S \in N$)
- R is a set of rules/productions of the form $X \rightarrow \gamma$
- P is a probability function
  - P: $R \rightarrow [0,1]$
  - A grammar G generates a language model L.

$$\sum_{\gamma \in T^*} P(\gamma) = 1$$

# PCFGs

| | | | |
|---|---|---|---|
| S → NP VP | 1.0 | N → *people* | 0.5 |
| VP → V NP | 0.6 | N → *fish* | 0.2 |
| VP → V NP PP | 0.4 | N → *tanks* | 0.2 |
| NP → NP NP | 0.1 | N → *rods* | 0.1 |
| NP → NP PP | 0.2 | V → *people* | 0.1 |
| NP → N | 0.7 | V → *fish* | 0.6 |
| PP → P NP | 1.0 | V → *tanks* | 0.3 |
| | | P → *with* | 1.0 |

# The probability of trees and strings

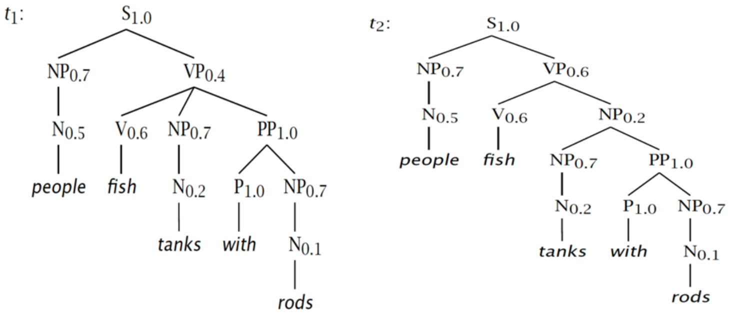


- $P(t)$ – The probability of a tree $t$ is the product of the probabilities of the rules used to generate it.
- $P(s)$ – The probability of the string $s$ is the sum of the probabilities of the trees which have that string as their yield

$P(s) = \sum_j P(s,t)$ where $t$ is a parse of $s = \sum_j P(t)$

# Frame Title





$t_1$:
$S_{1.0}$
$NP_{0.7}$
$VP_{0.4}$
$N_{0.5}$
$V_{0.6}$
$NP_{0.7}$
$PP_{1.0}$
people
fish
$N_{0.2}$
$P_{1.0}$
$NP_{0.7}$
tanks
with
$N_{0.1}$
rods
$t_2$:
$S_{1.0}$
$NP_{0.7}$
$VP_{0.6}$
$N_{0.5}$
$V_{0.6}$
$NP_{0.2}$
people
fish
$NP_{0.7}$
$PP_{1.0}$
$N_{0.2}$
$P_{1.0}$
$NP_{0.7}$
tanks
with
$N_{0.1}$
rods

# The probability of trees and strings

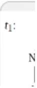
- $s$ = *people fish tanks with rods*
- $P(t_1) = 1.0 \times 0.7 \times \underline{0.4} \times 0.5 \times 0.6 \times 0.7 \times 1.0 \times 0.2 \times 1.0 \times 0.7 \times 0.1 = 0.0008232$

Verb attach

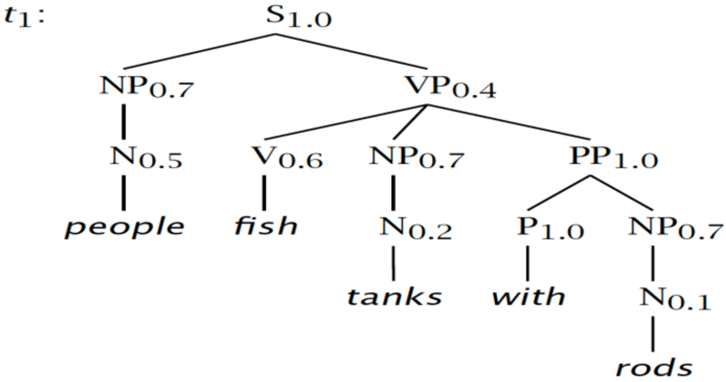
- $P(t_2) = 1.0 \times 0.7 \times \underline{0.6} \times 0.5 \times 0.6 \times \underline{0.2} \times 0.7 \times 1.0 \times 0.2 \times 1.0 \times 0.7 \times 0.1 = 0.00024696$

Noun attach

- $P(s) = P(t_1) + P(t_2) = 0.0008232 + 0.00024696 = 0.00107016$

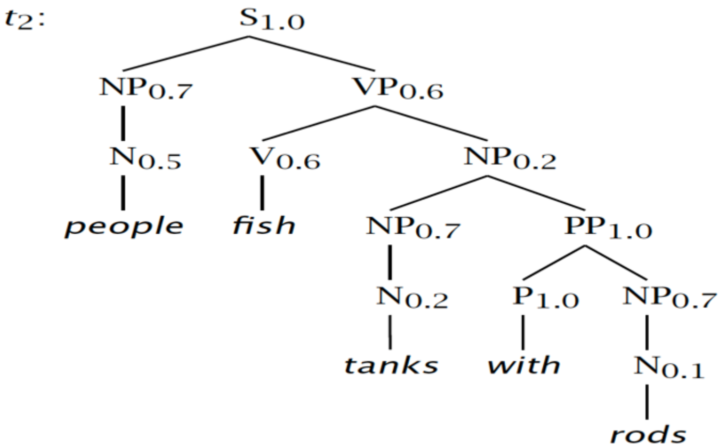


# The probability of trees and strings



# The probability of trees and strings

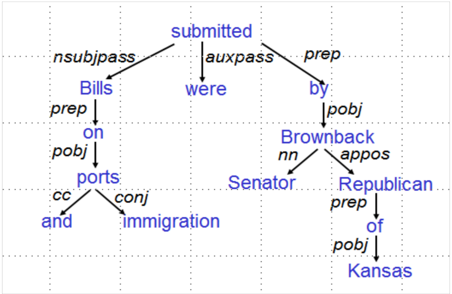




# Dependency relation



Dependency syntax postulates that syntactic structure consists of lexical items linked by binary asymmetric relations ("arrows") called dependencies

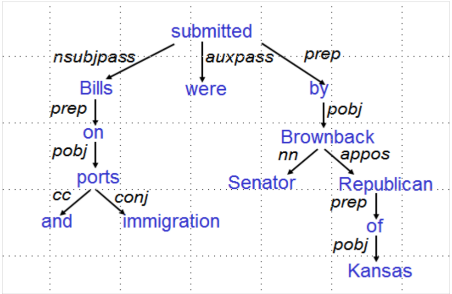



The arrows are commonly **typed** with the name of grammatical relations (subject, prepositional object, apposition, etc.)

# Dependency relation



Dependency syntax postulates that syntactic structure consists of lexical items linked by binary asymmetric relations ("arrows") called dependencies




The arrow connects a **head** (governor, superior, regent) with a **dependent** (modifier, inferior, subordinate)

Usually, dependencies form a tree (connected, acyclic, single-head)

# Dependency relation

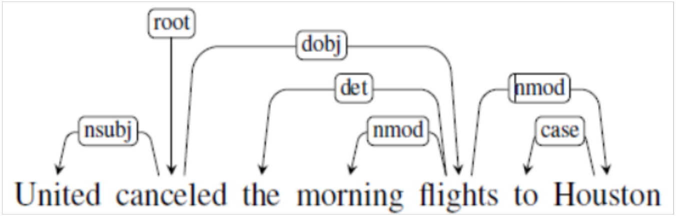


| **Clausal Argument relations** | **Description** |
|---|---|
| NSUBJ | Nominal subject |
| DOBJ | Direct object |
| IOBJ | Indirect object |
| CCOMP | Clausal Complement |
| XCOMP | Open clausal complement |
| **Nominal Modifier Relations** | **Description** |
| NMOD | Nominal modifier |
| AMOD | Adjectival modifier |
| NUMMOD | Numeric modifier |
| **Other Notable Relations** | **Description** |
| CONJ | Conjunct |

Selected dependency relations from the Universal Dependency set [4]

# Dependency Grammar and Dependency Structure

Example of dependency structure with universal dependency relations:




Clausal relations NSUBJ and DOBJ identify the subject and direct object of the predicate cancel, while NMOD, DET, and CASE relations denote modifiers of the nouns flights and Houston

Dependency structure is directed graph:
G = (V, A) V: set of vertices A: set of ordered pairs of vertices, We will refer A as arcs.

- V corresponds exactly to the set of words in the given sentence.
- A captures the head-dependent and grammatical function relationship between the elements in V

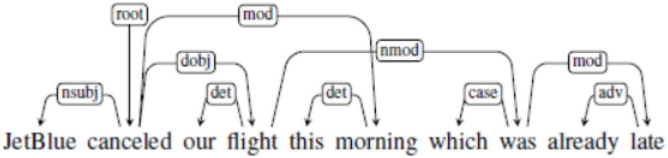
**Dependency Tree**

Dependency tree is directed graph that satisfies the following constraints:

1. There is single designated root node that has no incoming arcs.
2. With the exception of root node, each vertex has exactly one incoming arc.
3. There is a unique path from the root node to each vertex in V.

# Projectivity

- An arc from a head to a dependent is said to be projective if there is a path from the head to every word that lies between the head and the dependent in the sentence.
- A dependency tree is then said to be projective if all the arcs that make it up are projective.
- Consider the following example:

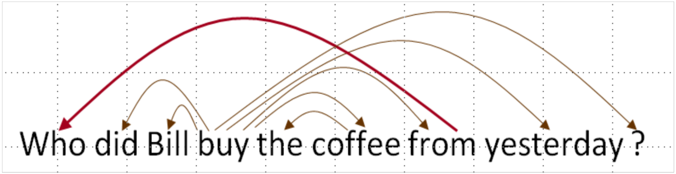



Arc from *flight* to its modifier *was* is non-projective, since there is no path from *flight* to intervening words *this* and *morning*.

# Projectivity



- Dependencies from a CFG tree using heads must be projective
  There must not be any crossing dependency arcs when the words are laid out in their linear order, with all arcs above the words.
- But dependency theory normally does allow non-projective structures to account for displaced constituents
  You can't easily get the semantics of certain constructions right without these non-projective dependencies

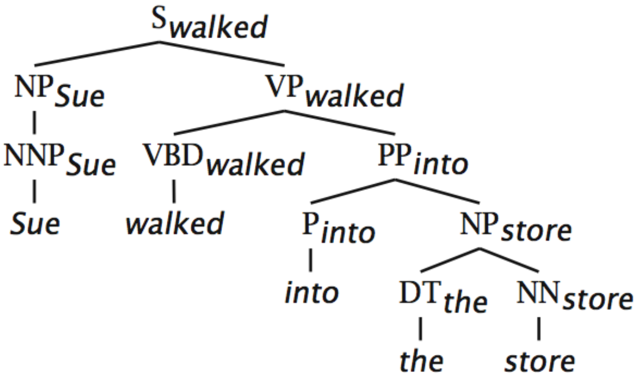


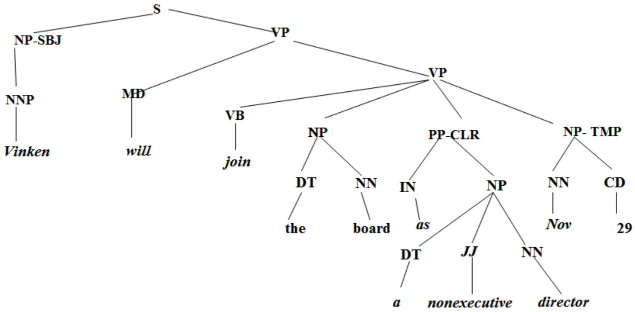
# Relation between phrase structure and dependency structure



- A dependency grammar has a notion of a head. Officially, CFGs don't.
- But modern linguistic theory and all modern statistical parsers (Charniak, Collins, Stanford, ... ) do, via hand-written phrasal "head rules":
  The head of a Noun Phrase is a noun/number/adj/...
  The head of a Verb Phrase is a verb/modal/...
- The head rules can be used to extract a dependency parse from a CFG parse
- The closure of dependencies gives constituency from a dependency tree
- But the dependents of a word must be at the same level (i.e., "flat") – there can be no VP!

# Relation between phrase structure and dependency structure (cont.)

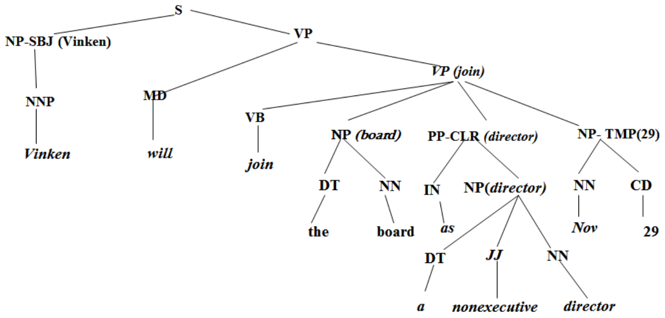





a) Example of a phrase structure "Vinken will join the board as a nonexecutive director Nov 29"

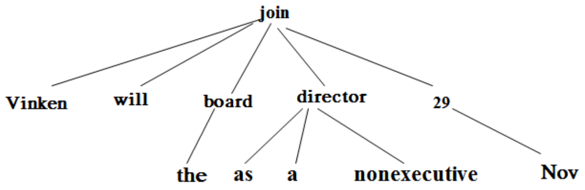
# Relation between phrase structure and dependency structure






b) To translate a) structure to dependency structure

# Relation between phrase structure and dependency structure

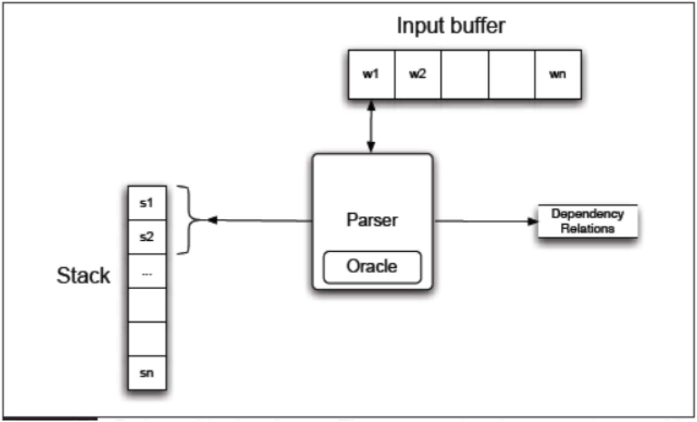



c) Dependency structure of sentence

# Transition – Based Dependency parsing

- Dependency parsing is motivated by a stack-based approach called **shift-reduce parsing**.
- **Configuration** consists of a stack, an input buffer of words, or tokens, and a set of relations representing a dependency tree.
- Parsing process consists of a sequence of transitions through the space of possible configurations.

# Basic transition- based dependency parser




Basic transition-based dependency parser

# MaltParser [Nivre et al. 2008]

- A simple form of greedy discriminative dependency parser
- The parser does a sequence of bottom-up actions
  Roughly like "shift" or "reduce" in a shift-reduce parser, but the "reduce" actions are specialized to create dependencies with head on left or right
- The parser has:
  - a stack $\delta$, written with top to the right. which starts with the ROOT symbol
  - a buffer $\beta$, written with top to the left. which starts with the input sentence
  - a set of dependency arcs A. which starts off empty
  - a set of actions

# Basic transition-based dependency parser

**Start:** $\sigma = [\text{ROOT}],\ \beta = w_1, \ldots, w_n,\ A = \emptyset$

1. Shift $\quad \sigma,\ w_i|\beta,\ A \rightarrow \sigma|w_i,\ \beta,\ A$
2. Left-Arc$_r$ $\quad \sigma|w_i,\ w_j|\beta,\ A \rightarrow \sigma,\ w_j|\beta,\ A \cup \{r(w_j, w_i)\}$
3. Right-Arc$_r$ $\quad \sigma|w_i,\ w_j|\beta,\ A \rightarrow \sigma,\ w_i|\beta,\ A \cup \{r(w_i, w_j)\}$

**Finish:** $\beta = \emptyset$

Notes: Unlike the regular presentation of the CFG reduction step, dependencies combine one thing from each stack and buffer

# MaltParser (cont.)



Actions ("arc-eager" dependency parser)

Start: $\sigma = [\text{ROOT}]$, $\beta = w_1, \ldots, w_n$, $A = \emptyset$

1. Left-Arc$_r$ $\quad \sigma|w_i, w_j|\beta, A \rightarrow \sigma, w_j|\beta, A \cup \{r(w_j, w_i)\}$

   Precondition: $r'(w_k, w_i) \notin A$, $w_i \neq \text{ROOT}$

2. Right-Arc$_r$ $\quad \sigma|w_i, w_j|\beta, A \rightarrow \sigma|w_i|w_j, \beta, A \cup \{r(w_i, w_j)\}$

3. Reduce $\quad \sigma|w_i, \beta, A \rightarrow \sigma, \beta, A$

   Precondition: $r'(w_k, w_i) \in A$

4. Shift $\quad \sigma, w_i|\beta, A \rightarrow \sigma|w_i, \beta, A$

Finish: $\beta = \emptyset$

This is the common "arc-eager" variant: a head can immediately take a right dependent before its dependents are found

# MaltParser (cont.)



| | | | |
|---|---|---|---|
| . | [ROOT] | [Happy, children, ...] | Ø |
| Shift | [ROOT, Happy] | [children, like, ...] | Ø |
| $LA_{amod}$ | [ROOT] | [children, like, ...] | {amod(children, happy)} = $A_1$<br>**(children → happy)** |
| Shift | [ROOT, children] | [like, to, ...] | $A_1$ |
| $LA_{nsubj}$ | [ROOT] | [like, to, ...] | $A_1$ ∪ {nsubj(like, children)} = $A_2$<br>**( like → children)** |
| $RA_{root}$ | [ROOT, like] | [to, play, ...] | $A_2$ ∪{root(ROOT, like) = $A_3$ |
| Shift | [ROOT, like, to] | [play, with, ...] | $A_3$ |
| $LA_{aux}$ | [ROOT, like] | [play, with, ...] | $A_3$∪{aux(play, to) = $A_4$<br>**( play → to)** |
| $RA_{xcomp}$ | [ROOT, like, play] | [with their, ...] | $A_4$∪{xcomp(like, play) = $A_5$<br>**( like → play)** |

# Example

Happy children like to play with their friends.

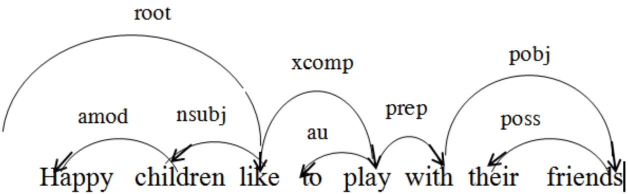
| | | | |
|---|---|---|---|
| $RA_{xcomp}$ | [ROOT, like, play] | [with their, …] | $A_4$U{xcomp(like, play) = $A_5$ |
| $RA_{prep}$ | [ROOT, like, play, with] | [their, friends, …] | $A_5$U{prep(play, with) = $A_6$ |
| Shift | [ROOT, like, play, with, their] | [friends, .] | $A_6$ |
| $LA_{poss}$ | [ROOT, like, play, with] | [friends, .] | $A_6$U{poss(friends, their) = $A_7$ |
| $RA_{pobj}$ | [ROOT, like, play, with, friends] | [.] | $A_7$U{pobj(with, friends) = $A_8$ |
| Reduce | [ROOT, like, play, with] | [.] | $A_8$ |
| Reduce | [ROOT, like, play] | [.] | $A_8$ |
| Reduce | [ROOT, like] | [.] | $A_8$ |
| $RA_{punc}$ | [ROOT, like, .] | [] | $A_8$U{punc(like, .) = $A_9$ |

You terminate as soon as the buffer is empty. Dependencies = $A_9$

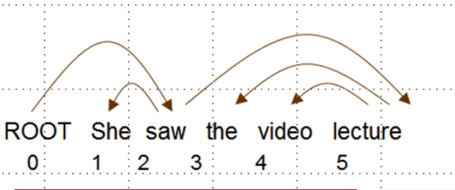
# Example



Example: Happy children like to play with their friends



# Evaluation of Dependency Parsing




$$Acc = \frac{\text{\# correct deps}}{\text{\# of deps}}$$

UAS = 4 / 5 = 80%

LAS = 2 / 5 = 40%

**Gold**

| | | | |
|---|---|---|---|
| 1 | 2 | She | nsubj |
| 2 | 0 | saw | root |
| 3 | 5 | the | det |
| 4 | 5 | video | nn |
| 5 | 2 | lecture | dobj |

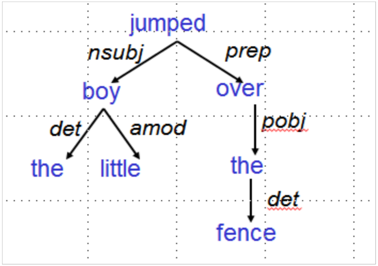
**Parsed**

| | | | |
|---|---|---|---|
| 1 | 2 | She | nsubj |
| 2 | 0 | saw | root |
| 3 | 4 | the | det |
| 4 | 5 | video | nsubj |
| 5 | 2 | lecture | ccomp |

# Stanford Dependencies [de Marneffe et al. LREC 2006]

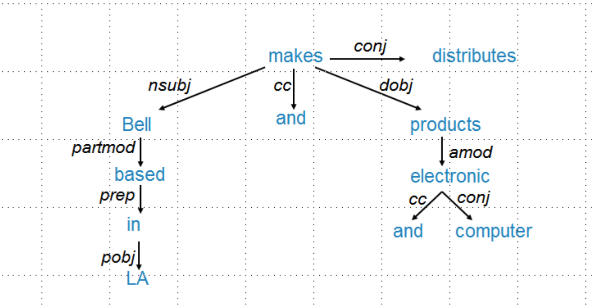


- The basic dependency representation is projective
- It can be generated by postprocessing headed phrase structure parses (Penn Treebank syntax)
- It can also be generated directly by dependency parsers, such as MaltParser, or the Easy-First Parser



# Graph modification to facilitate semantic analysis

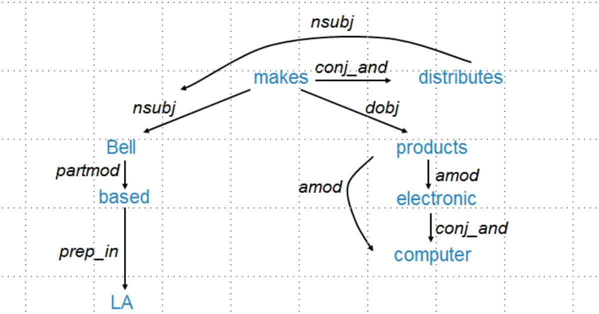
Bell, based in LA, makes and distributes electronic and computer products.



# Graph modification to facilitate semantic analysis



Bell, based in LA, makes and distributes electronic and computer products.



# REFERENCE OF CHAPTER 3




1. http://www.sfs.uni-tuebingen.de/∼dm/10/ss/dep/dg-slides-2x2.pdf
2. https://web.stanford.edu/∼jurafsky/NLPCourseraSlides.html
3. Speech and Language Processing. Daniel Jurafsky & James H. Martin. Copyright 2018.
4. https://universaldependencies.org/u/dep. Access: Jan 2023.

*Thank you!*